Assignment 2

Team number: 8
Team members

Name	Student Nr.	Email
Alejandro Guerena Gonzalez	2727241	a.guerenagonzalez@student.v u.nl
Jakub Dryja	2732653	j.o.dryja@student.vu.nl
Daniele Roggi	2735539	d.roggi@student.vu.nl
Salvatore Pernice	2722134	s.pernice@student.vu.nl

Do NOT describe the obvious in the report (e.g., we know what a `name` or `id` attribute means), focus more on the **key design decisions** and their "**why**", the pros and cons of possible **alternative designs**, etc.

Format: establish formatting conventions when describing your models in this document. For example, you style the name of each class in bold, whereas the attributes, operations, and associations as underlined text, objects are in italic, etc.

Summary of changes from Assignment 1

Author(s): Alejandro Guerena Gonzalez, Daniele Roggi

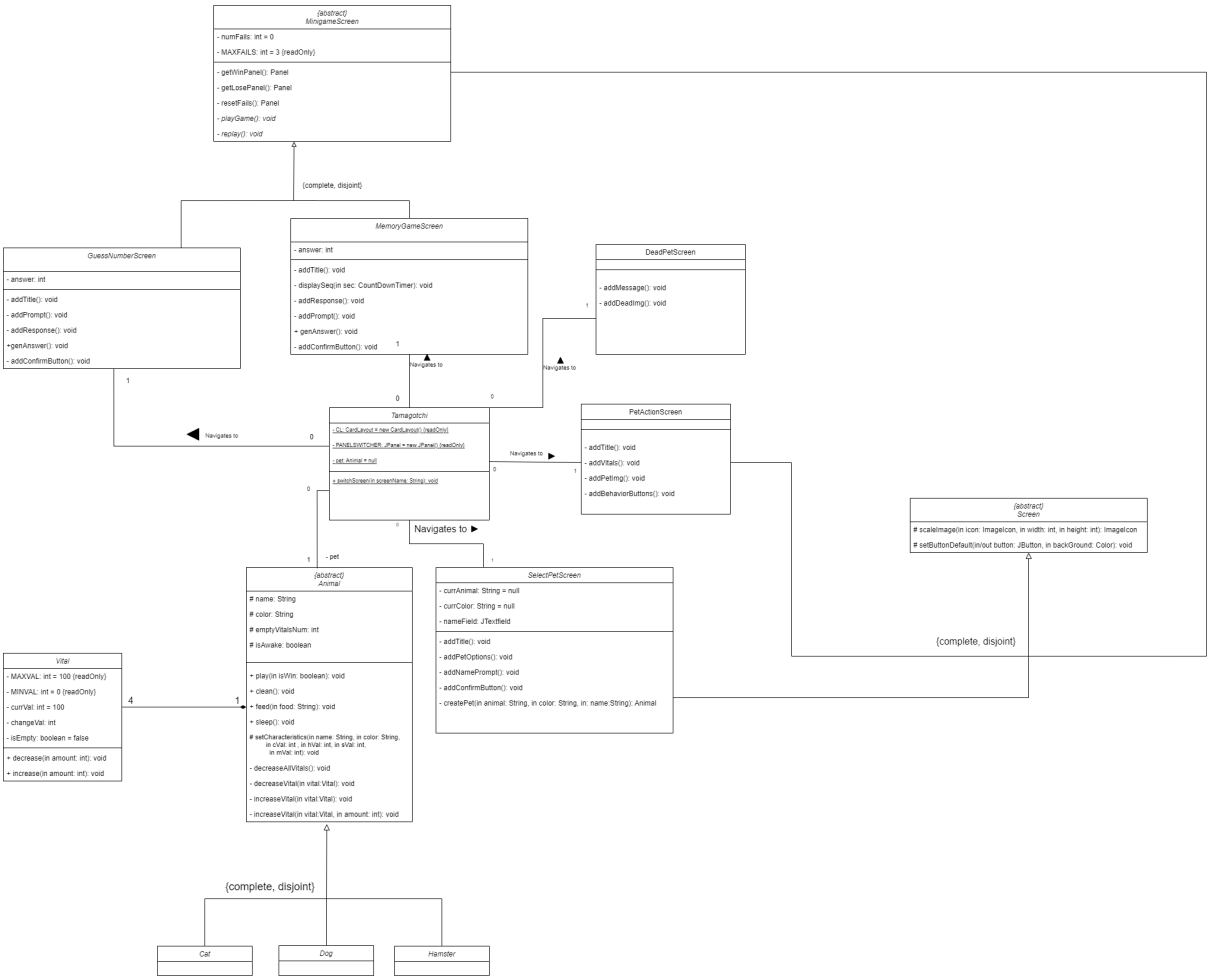
- Stakeholders are further described with reasoning.
- Changed wording of creature to animal for consistency.
- Additional and more specific description on the following functional features with the user of moscow format and reasoning:
    - Create a new Animal(F1)
    - Keep track of vitals (F2)
    - Sleep (F3)
    - Play (F4)
    - Feed (F5)
    - Clean (F6)
    - Animal dies upon low vitals (F7)
    - Memory Game (F8)
    - Guess the number (F9)
    - Play again (F10)
- Additional description on the following quality requirements with the user of moscow format and reasoning:
    - Vitals work properly (QR1), merged with previous QR2
    - Simple interactions with pets (QR2)
    - Responsive activities and games (QR3)
    - Extendable features (QR4)

- Additional functional features
    - F10

Class diagram

Author(s): Alejandro Guerena Gonzalez

This chapter contains the specification of the UML class diagram of your system, together with a textual description of all its elements.



Tamagotchi

The Tamagotchi class represents the application itself hence why Tamagotchi is a subclass of JFrame. The role this class serves is to switch between different screens(SelectPetScreen, PetActionScreen, GuessNumberScreen, MemoryGameScreen and DeadPetScreen) of the GUI. This is why there is binary association with every screen of the application in order to navigate to the screen. Each instance of Tamagotchi has exactly one instance of everyscreen. However, each instance of a screen does not have any instance of the Tamagotchi class since there should only ever be one application running. The screens are able to communicate without an instance of Tamagotchi class through a public static function of switchScreen. The name of the screen is passed through the function parameter then the panel of the frame is set to the respective screen through the

cardlayout attribute CL. The Animal instance is an attribute of the Tamagotchi class as other screens may retrieve the pet object.

SelectPetScreen

SelectPetScreen represents the screen where the user is able to create the animal. The operations are shown as private as these operations will be called when an instance of SelectPetScreen is created (this is common through every instance of a screen in this project). addPetOptions() is an operation that adds cat, dog and hamster as options as radio buttons as well as colour options as radio buttons onto the screen. Everytime the user presses a radio button, the currAnimal attribute or currColor attribute changes to the animal or colour as a string. This is done so that only one instance of Animal is created when the user presses confirm. Additionally, these attributes are necessary in case the user presses a colour radio button before an animal radio button, where setting the animal's colour is not possible. The only operation that is not called when an instance of SelectPetScreen is called is the createPet operation. Only when the user has pressed the confirm button and has a valid selection, then createPet is called with the corresponding attributes. Previous editions of the class diagram included an overloaded function of switchScreen with additional parameters to pass pet. However, it is better practice to use a setter operation and then simply call switchScreens function.

{abstract} Animal

Animal is an abstract class because it represents a generic animal. A Tamagotchi game can not simply have an animal as a pet, the pet must be a specific animal. Animal being a parent class allows to alleviate redundant code that subclasses might share for example the play() operation that is a public operation a screen may call to increase vitals after the user presses play. This design choice to utilise encapsulation allows other classes to interact with an Animal without having to know the state or attributes of the animal. Decreasing all vitals is another operation in Animal class that is used to decrease all vitals periodically. IncreaseVital is an operation that is overloaded for the case of feeding for instance where the vital is increased based on the user's choice. Animal has several attributes to describe the state such as emptyVitalsNum which holds a number to indicate how many vitals are empty, this is necessary for the option to switch screens to the DeadPetScreen. Each instance of Tamagotchi has exactly one instance of an Animal named pet.

Cat, Dog, Hamster

The Cat, Dog, Hamster are subclasses of Animal that represent a pet someone may have. The subclasses inherit all the attributes and operations of Animal class. Although the subclasses in the class diagram are empty, each subclass of Animal is unique by the parameter that is passed to Characteristics. The values that are passed when the Characteristics operation is called assigns the rate at which each vital depletes. This occurs when any subclass instance of Animal is created. The relationship between Animal and the subclasses is complete because each instance of Animal must be an instance of at least one of the subclasses of Animal (Cat, Dog or Hamster). Disjoint because an object may be an instance of a maximum of one subclass of Animal.

Vital

Vital is a class that represents a generic vital. Every vital has the same implementation besides the rate at which the vital depleds. Contrary to a screen subclass, all of the operations are public because they are called after instantiated. The minimum value and maximum value of the vitals are constants 0-100 and a new vital starts at value 100. The attribute changeVal is the rate at which the vital increases or decreases. Creating a subclass to vital for each vital is possible however there is only one value that is set when initialising a vital. This would be similar to our abstract Animal class, though the difference between animals in Tamagotchi is arguably more significant than differences in vital bars. Specifically, there are far more options to provide unique implementation between each animal than a vital bar that are similar besides one value. The abstract Animal class is composed of 4 vital classes. This is because the vitals are unable to exist on their own without the Animal class.

PetActionScreen

PetActionScreen represents the screen where the user is able to perform certain actions or behaviours of a pet. The behaviours of play, clean, feed and sleep are displayed through the addBehaviorButtons operation. All of the operations in PetActionScreen are private because they are called when an instance of PetActionScreen is created. All of the operations add GUI components onto the screen. Separating sections where components are added to the screen make the code easier to read and understand. There are no attributes to PetActionScreen, as the screen does not have any state, the Animal does. Having every screen be an individual class allows for modular progress as well as clarification to the Tamagotchi class for being able to switch between screens.

{abstract} MinigameScreen

MiniGameScreen represents a screen for a general minigame. Some operations such as getWinPanel, getLosePanel or resetFails may be the same for all minigame screens, thus subclasses may use these operations to reuse code. PlayGame however, is an operation that would be specific to the game's logic. This is why the playGame operation is italicised to represent an abstract operation. Replay is also an abstract operation, since different games hold different logic, to restart the game must be specific to a certain type of game. Since our games could include a maximum number of attempts, the screens would then have a state depending on how many fails the user has and the maximum number of attempts. A MinigameScreen is also a Screen thus it is a subclass of Screen which inherits the utility operations as well.

GuessNumberScreen

GuessNumberScreen represents the screen that displays the minigame where the user attempts to guess the right number in order to increase the mood vital at the expense of the energy vital. When an instance of GuessNumberScreen is created, several GUI components are displayed on the screen as well as an answer. The operation to generate a new answer is public because the Tamagotchi class should be able to access this operation without having to create a new instance of the class every time. After confirming an answer, the response is displayed. This class is able to interact with the pet through Tamagotchi class depending on if the user wins or not.

MemoryGameScreen

The MemoryGameScreen represents the screen that displays the minigame where the user attempts to repeat a displayed number sequence back. This is done firstly by generating a new answer to the game, then displaying several GUI components to the screen such as the number sequence for a brief period of time. The user is able to attempt to repeat the sequence through the text field provided in addPrompt operation. Once confirmed, the response displays whether the user was correct or not. Each instance of MinigameScreen must be an instance of at least one of the subclasses of Minigame, hence the complete relationship. The relationship is also disjoint because an object may be an instance of a maximum of one subclass of MinigameScreen since an instance of a screen is created only once.

DeadPetScreen

The DeadPetScreen represents the screen that displays an image of their dead pet without other means of interacting with the application besides closing the frame. A title/message to display to the user that their pet has died due to the specified vitals being empty.

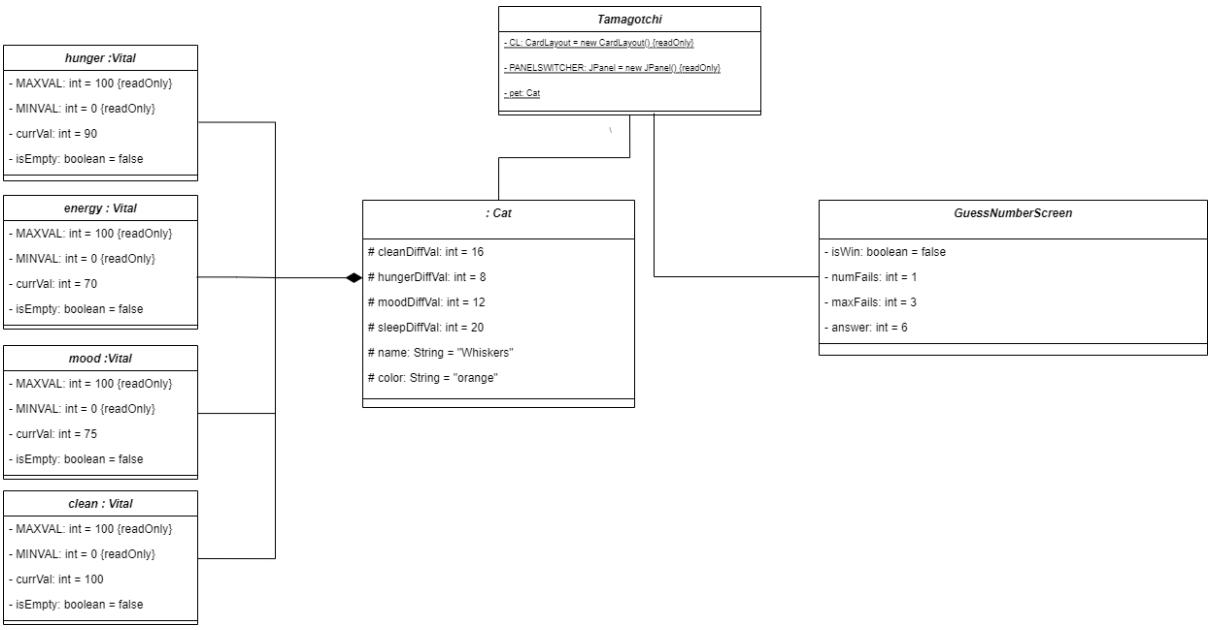
{abstract} Screen

The Screen class is an abstract class because the class represents a generic screen. The abstract class has two operations, one is to scale an image to a certain dimension and the other operation sets default attributes of a button. When implementing a GUI in java, it becomes clear that there are several redundant lines of code for each setting attribute of every component instance. This becomes a problem as a reader because it becomes significantly more difficult to understand. The abstract screen class solves this problem by having each screen subclass instance be able to utilise these operations. The use of screen in the name of every subclass of Screen makes this design choice clear and helpful to the reader with insignificant added complexity. That being said, one downside to this choice is that the default values may not be apparent from the first moment. The relationship between the subclasses of Screen is complete because each instance of Screen must be an instance of at least one of the subclasses of Screen (SelectPetScreen, PetActionScreen, GuessNumberScreen, MemoryGameScreen or DeadPetScreen). Disjoint because an object may be an instance of a maximum of one subclass of screen. This is a class that likely will present itself more useful later in the development stage, for example a generic title dimension operation may be included with parameters of a JLabel.

Object diagram

Author(s): Salvatore Pernice

This chapter contains the description of a "snapshot" of the status of your system during its execution. This chapter is composed of a UML object diagram of your system, together with a textual description of its key elements.



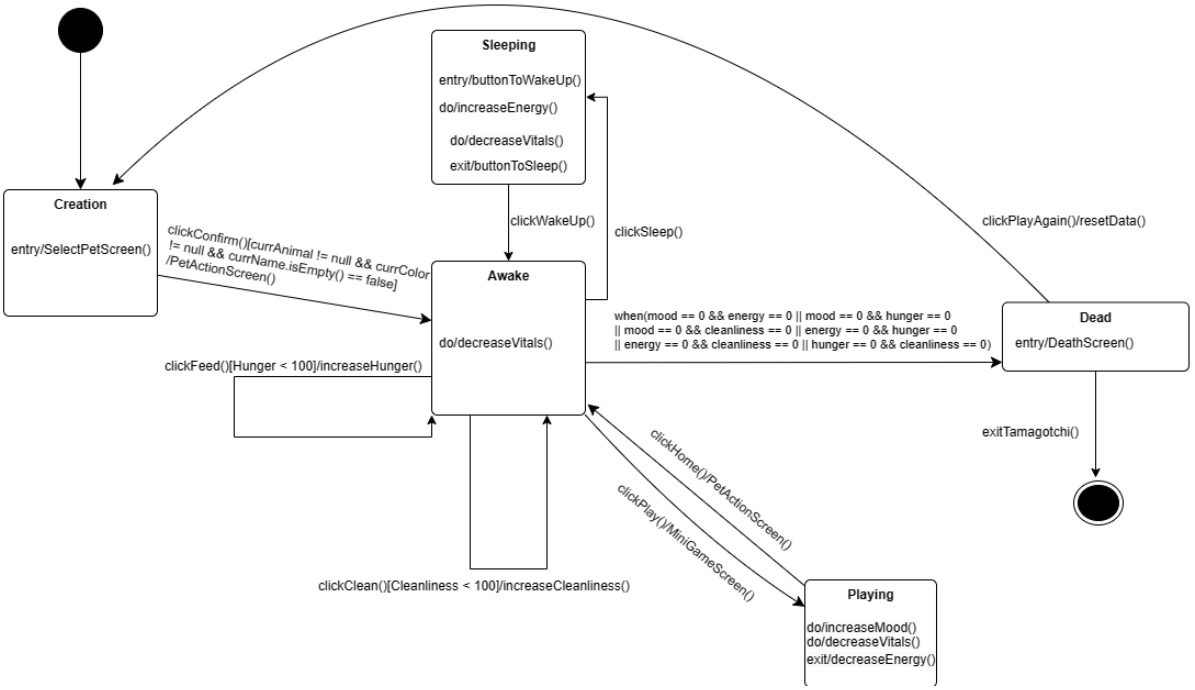
The current snapshot of the system represents a state in which the user is playing the MiniGame *"*Guess the number". The *PANELSWITCHER* JPANEL(Java Swing Package) in the Tamagotchi *main* class has been set to display GuessNumberScreen, hence all the other subclasses of abstract class "Screen" from the class diagram can be omitted in this diagram as only one panel can be displayed at a time for our program.

The user is currently playing the Minigame and has failed once in guessing the number since the start of the game. The number generated randomly is "6" and the user has two more attempts available to guess the number and win the game.

The snapshot obviously includes the living creature itself, instantiated as a *cat* by the user, and its set name and colour (the remaining "diff" values are characteristics of the animal and don't depend on the user). The current values of the 4 vitals of the animal can also be seen in "currVal" under the respective *vital*. These values can decrease with time or can increase if actions such as minigames, feed, clean and sleep are performed. **The vitals also decrease while a game is being played. Playing minigames, or any action for the matter, does not set the vitals in a "freeze" state** .

State machine diagrams

Author(s): Daniele Roggi



Link: [State Machine Diagram 1 - diagrams.net]

This state machine represents the class Tamagotchi together with various classes related to it such as PetActionScreen, SelectPetScreen, Animal and DeathScreen. This diagram is used to demonstrate the general functioning of the Tamagotchi by analysing the various states in which a pet can be, from the creation to the death. This diagram also illustrates the basic principles of the Tamagotchi by examining how actions performed by the player affect the vitals and the states of the pet..

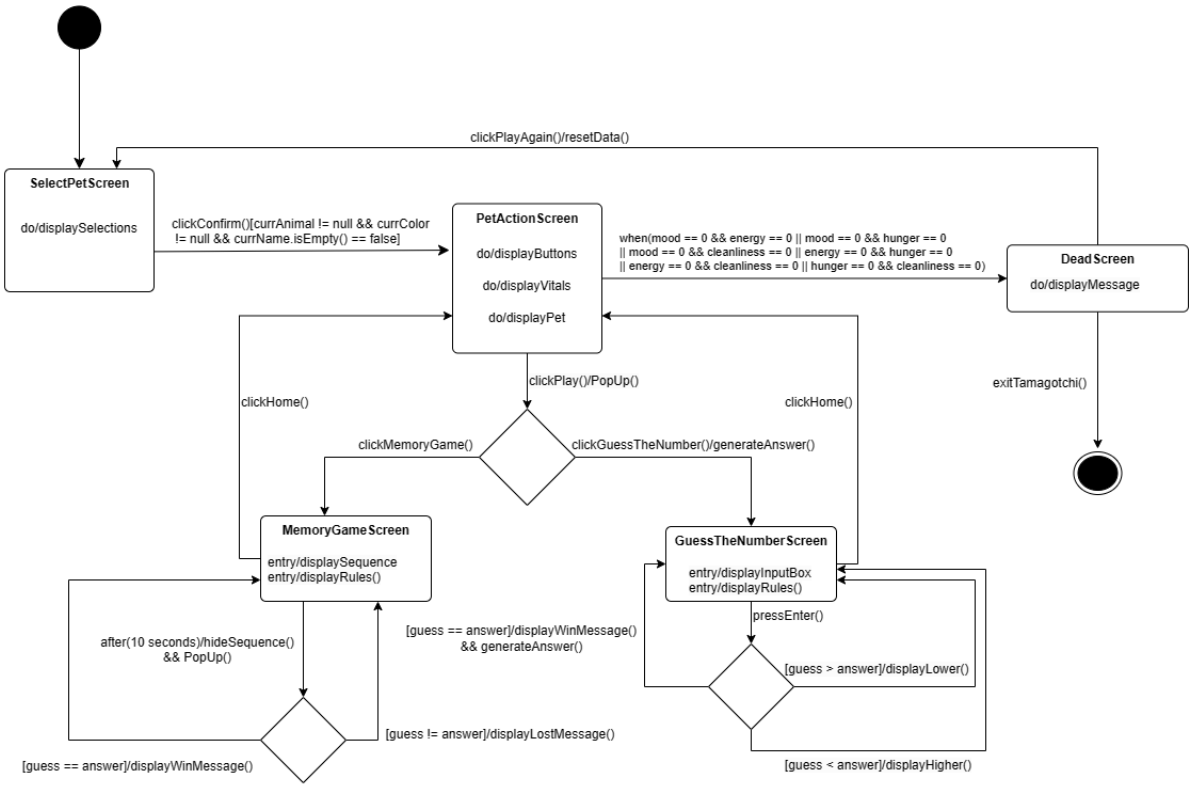
To begin with, the initial node transits to the ‘Creation’ state, where we enter the screen to personalise our pet, the trigger event is clicking on the Confirm button, if the conditions are met, a transition to the ‘Awake’ state takes place and the player enters the PetActionScreen, otherwise, the player is going to stay in the creation phase, he/she is going to have to fulfil the parameters and trigger the event again in order to exit this state.

Once inside the ‘Awake’ state, the vitals are visible and decrease with time while we stay in this state. A large number of events are now possible, to start with, if the player clicks on the Sleep button, a transition to the ‘Sleeping’ state takes place. Inside the latter, firstly the button switches from “Sleep” to “WakeUp”, then the energy vital continually increases until the player clicks on the WakeUp button, which triggers the transition back to ‘Awake’, the button reverts back to “Sleep”.

Moreover, the events clickFeed() and clickClean() can also be triggered inside ‘Awake’, which represent clicking on the Feed and Clean buttons respectively, both individually increase their respective vital, provided that they are not already at their maximum score.

Furthermore, the player can trigger the event ClickPlay() by clicking on the Play button, this will trigger a transition into the ‘Playing’ state while switching to minigames screen. The playing state of the pet is generalised as increasing the mood vital and decreasing the energy vital, in practice, the minigames screen will offer more events and interactions. When the player triggers the clickHome() event by clicking on the Home button after finishing a game, the tamagotchi reverts back to the ‘Awake’ state while switching back to the PetActionScreen.

To conclude with, when two vitals reach the value 0, a transition takes place into the ‘Dead’ state. The player has now the opportunity to start a new game by clicking the New Game button, which will trigger a transition from the ‘Dead’ to the ‘Creation’ state, while deleting previous data. The player can in alternative simply decide to stop playing and exit the tamagotchi.
Comment: It can be confusing to see both decreasing and increasing vitals inside the Playing and Sleeping states, the reason behind this is that the vitals should always decrease with time, and when we are in a certain state, one or more vitals could increase or decrease even more, depending on the state.



Link: [state machine - diagrams.net]

This state machine represents the classes related to screens such as SelectPetScreen, PetActionScreen, DeathScreen and the two MiniGames screens. This diagram is used to analyse the behaviours of the screens used in our interface, relative to the actions performed by the player. The diagram also gives an overview on the functionalities of the two minigames.
To begin with, the player is inside the ‘SelectPetScreen’ state where he/she indicates the preference in the creation of the pet. Once the confirm button is clicked and the conditions are met, a transition into the ‘PetActionScreen’ state occurs.
Inside the latter state, the buttons, vitals and pet are displayed on the screen. The player can decide to enter a minigame by clicking on the Play button, which will trigger the event ClickPlay(), exiting the ‘PetSelectionScreen’ state. The player now has to choose between the two minigames being prompted on the screen, the choice made will trigger a transaction into the relative state.
In case the player chooses the GuessTheNumber minigame, an answer for the game is generated and we enter the ‘GuessTheNumberScreen’ state. Once inside the new state, an

input box together with some rules are displayed, entering a number and pressing enter triggers a transition outside the ‘GuessTheNumberScreen’ state. The answer given by the player is confronted with the previously generated number, different actions are performed relative to the two integers. In case the player guesses the number, a message such as ‘You won” is displayed and a new number is generated. Conversely, if the player does not guess the right number, the message “Lower” or “Higher” is displayed relative to the guess being made. Whichever the answer, after every guess, we go back to the ‘GuessTheNumberScreen’ state. Once the minigame has ended, the user can go back to the ‘PetActionScreen’ state by clicking the Home button.

In the other case, where the player chooses to play the MemoryGame, the transition takes place to the ‘MemoryGameScreen’ state. A sequence of 5 numbers is initially displayed together with some rules, after 10 seconds, the state is exited, the sequence is hidden and a popup appears asking the user for the answer. If the user answers correctly, a “You won” message is displayed and we go back inside the previous state, conversely, if the player answers wrongly, a “You lost” message is displayed and the game restarts. Once the minigame has ended, the user can go back to the ‘PetActionScreen’ state by clicking the Home button.

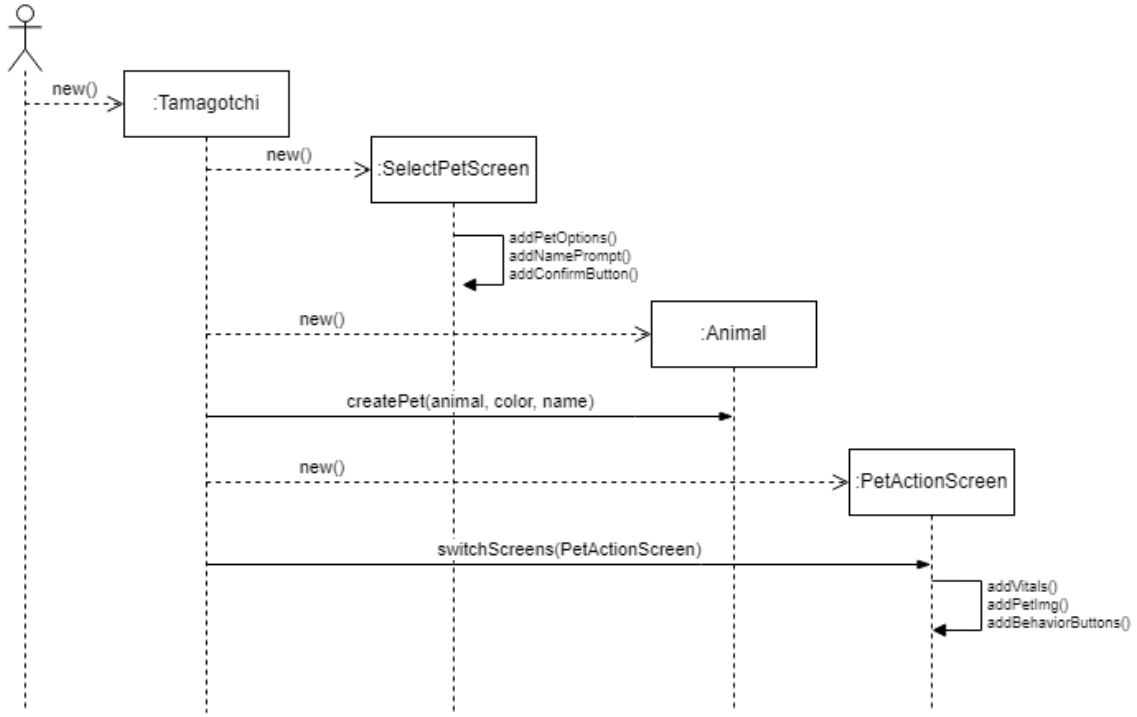
To conclude with, when two vitals reach the value 0, a transition takes place into the ‘DeadScreen’ state. The player has now the opportunity to start a new game by clicking the New Game button, which will trigger a transition from the ‘DeadScreen’ to the‘PetSelectionScreen’ state, while deleting previous data. The player can in alternative simply decide to stop playing and exit the tamagotchi.

Sequence diagrams

Author(s): Jakub Dryja

Pet Creation - Sequence Diagram

This sequence diagram models the process of creating a pet in the MyTamagotchi app. The goal of the diagram is to illustrate the interaction between the user and the app's system for creating and customising pets.



When the user launches the application, instances of the Tamagotchi and SelectPetScreen classes are created. The SelectPetScreen is responsible for presenting the user with options for selecting and customising a pet. Helper functions, such as addPetOptions(), addNamePrompt(), and addConfirmButton(), are called to set up the graphical interface of the SelectPetScreen.

Once the user has selected and customised their pet, they can confirm their choices by pressing the confirm button. When the confirm button is pressed, the createPet() method is called, and an instance of the Animal class is created with the appropriate attributes depending on the user's choices. A subclass of the Animal class is created to represent the selected pet.

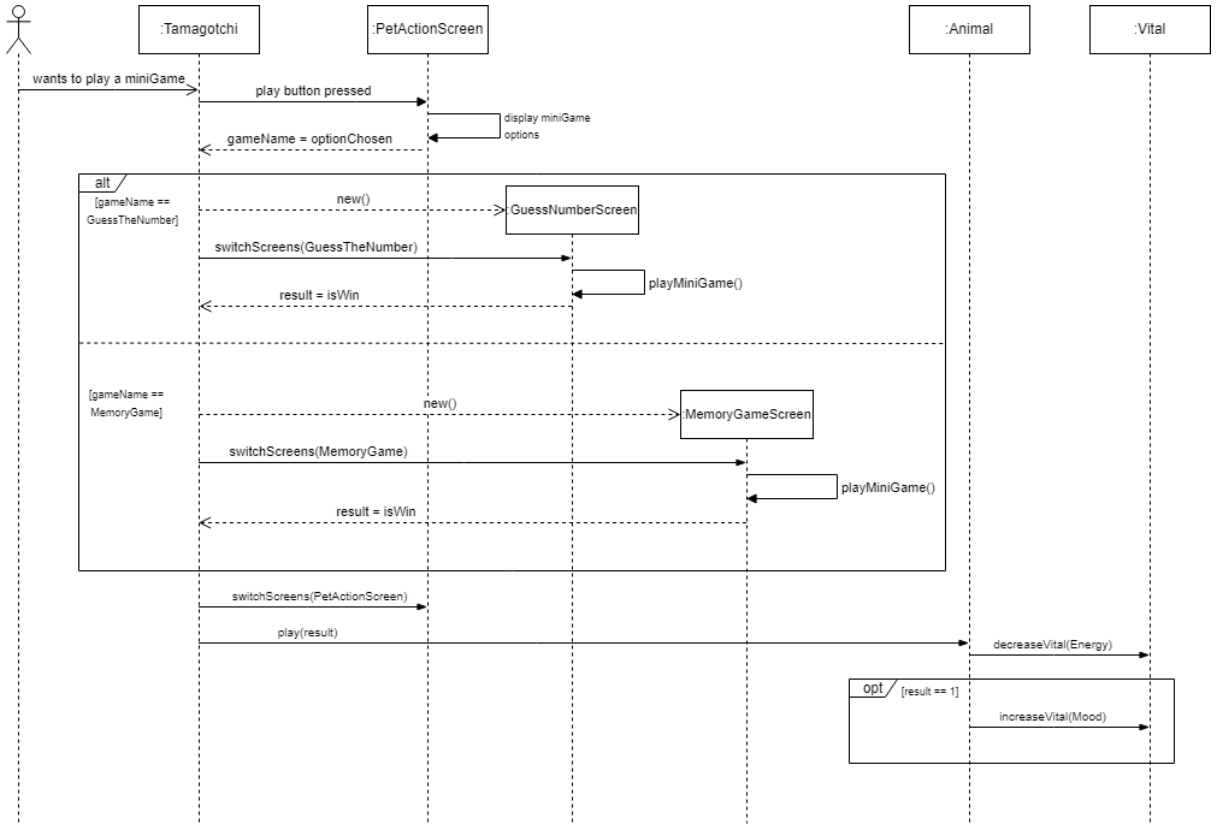
Next, as the user confirmed that he wants to continue, the app creates a new instance of the PetActionScreen and calls the switchScreens() method to switch to that screen. Helper functions are called to set up the graphical interface of the PetActionScreen.

The PetActionScreen is responsible for presenting the user with options for interacting with their pet, such as feeding, playing, and sleeping.

Throughout this process, the main class Tamagotchi manages the interaction between the various screens and classes involved in creating and customising the pet. The Tamagotchi class is responsible for creating instances of the screens(SelectPetScreen, PetActionScreen) and Animal classes.

Choosing a mini game and their impact on vitals

This sequence diagram depicts the process of selecting a mini game in the MyTamagotchi application and how it affects the vitals of the pet based on the game result. The objective of the diagram is to demonstrate the interaction with the vitals.



When the user decides to play a mini game with their pet, they need to press the play button in the PetActionScreen. This action displays a pop-up screen containing all possible mini games to choose from. As indicated in the diagram, the selection of a specific mini game results in the creation of a different subclass of the minigame. The Tamagotchi class then calls the switchScreens() method to switch to the mini game screen. Once the screen is switched, the mini game can be played. It is important to note that each mini game is played on a new screen, and thus, a separate sequence diagram is recommended for each of them to describe the user interaction during key parts of the games.

After the mini game ends, or the user chooses to finish the game, the result in the form of an isWin attribute is passed to the Tamagotchi class. This attribute indicates whether the mini game was won or lost, as all of the mini games are win/lose games. As a result, the screen switches back to the PetActionScreen using the switchScreens() method.

To impact the respective vitals, the play() method in the Animal class with the result of the mini game as a parameter is called. As illustrated in the diagram, regardless of the game result, the energy vital bar is decreased by an appropriate value (depending on the specific pet rates) by calling the decreaseVital() method with Energy as a parameter. In the case of a win, the mood vital bar is increased by calling the increaseVital() method with Mood as a parameter.

Time logs

https://docs.google.com/spreadsheets/d/1makaMgH1aErOeWJ_MhrNb1SC_dxbZ_df8Vcnu5VfyPU/edit?usp=sharing

Team number	8		
Member	**Activity**	**Week number**	**Hours**
Team	Discussion about diagrams and proper implemention in terms of general idea of the project	2	3
Salvo	Create class diagram draft	3	1
Alex	Create class diagram draft	3	1
Jakub	Create class diagram draft	3	1
Daniele	Create class diagram draft	3	1
Alex	Structure program for working dynamic UI system	4	7
Jakub	Implement the UI of PetActionScreen	4	5
Daniele	Prepare sketch for state machine diagram	4	3
Alex	Create complete second draft class diagram	4	3
Jakub	Prepare sketch for sequence diagram	4	2
Alex	Improve assignment 1	4	2
Alex	Explain the class diagram for assignment 2	4	5
Daniele	Improve assignment 1	4	2
Daniele	Create complete state machine diagrams for Assignement 2	5	4
Daniele	Describe completed state machine diagrams for Assignement 2	5	2
Jakub	Create complete sequence diagrams for Assignement 2	5	4
Jakub	Describe completed sequence diagrams for Assignement 2	5	2
Salvo	Create complete object diagram for Assignement 2	5	4
Salvo	Describe completed object diagram for Assignement 2	5	2